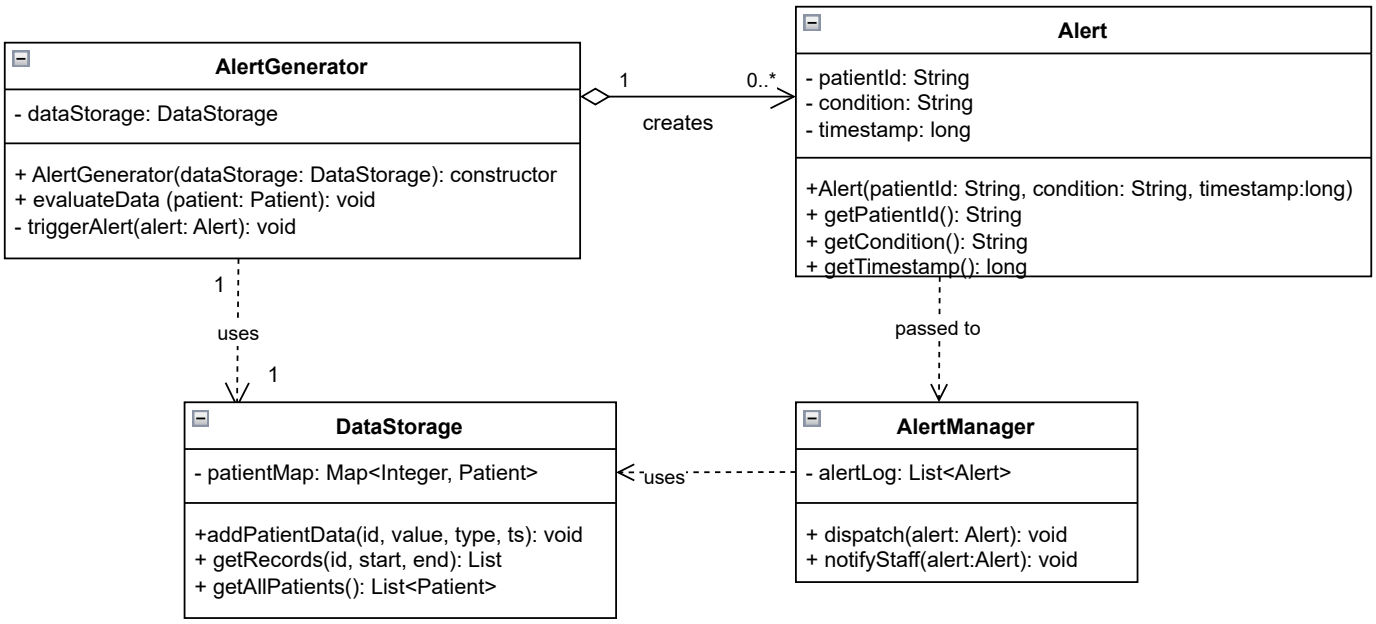
Alert Generation System



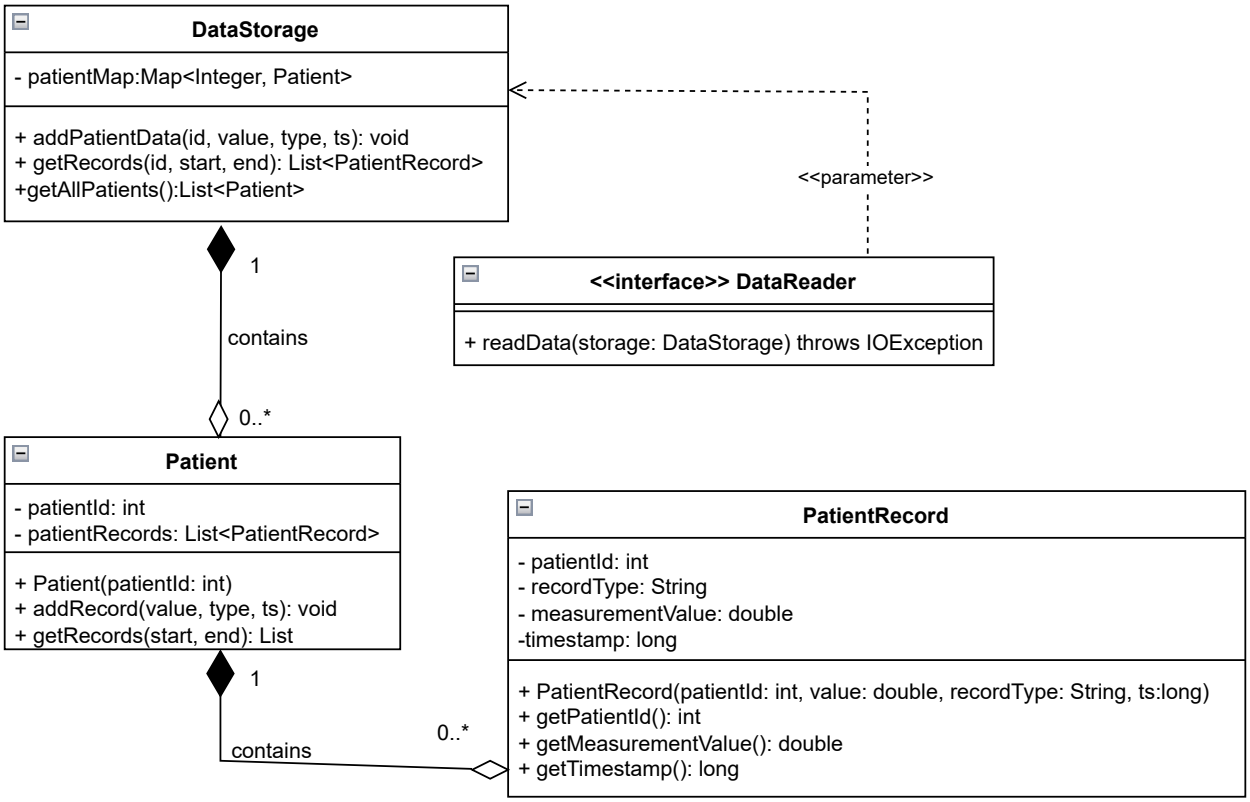
This diagram shows how the system detects problems with patient health and raises alerts.

The main class is AlertGenerator. It has access to DataStorage so it can read patient data. When it finds something wrong like an abnormal heart rate or low blood saturation, it calls `triggerAlert`, which creates an Alert object. Alert just holds three things: the patient ID, what the condition is, and when it happened. It doesn't do anything else.

AlertManager was added in to represent the idea that something should receive the alert and actually do something with it, like notifying a nurse or logging it. Keeping this separate from AlertGenerator means the two classes don't depend on each other more than necessary. One creates alerts, the other handles them.

The <<creates>> relationship means one AlertGenerator can create several alerts over time. DataStorage is included because AlertGenerator can't work without it.

DataStorageSystem



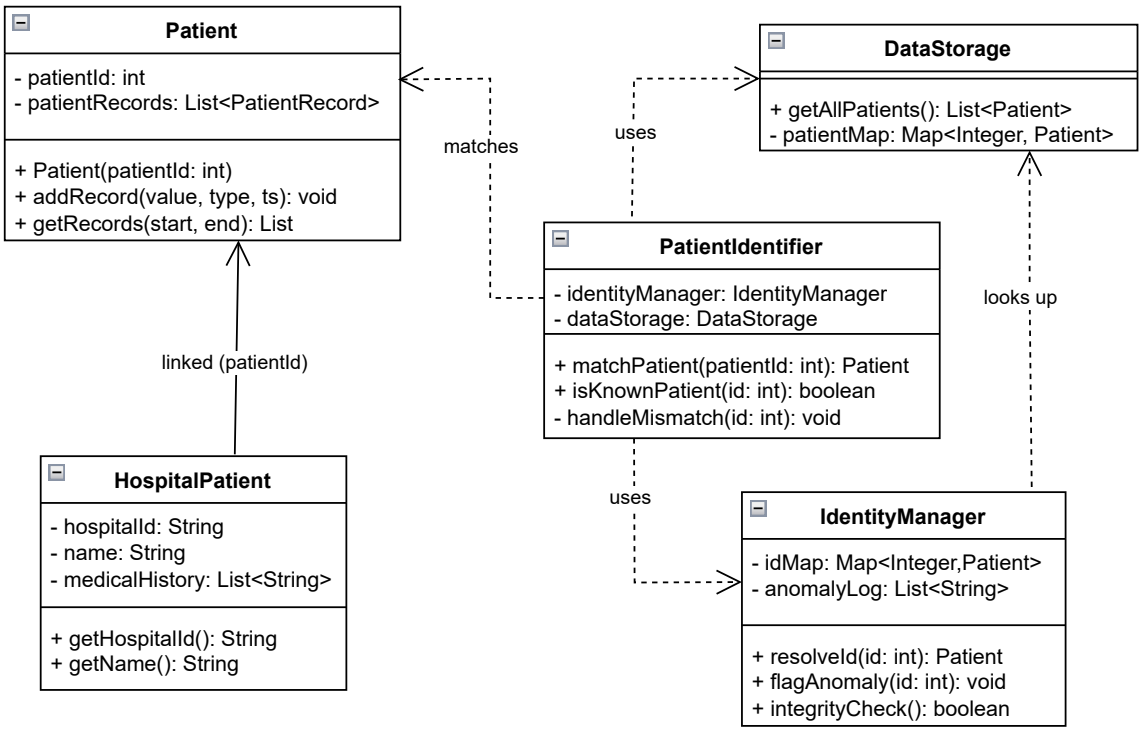
This diagram shows how patient data is stored and retrieved. DataStoage is the main class; it holds all patients in a map and lets other parts of the system add or fetch records. It has a composition relationship with Patient, meaning patients are created and managed by the storage.

Patient stores a list of PatientRecords objects. When addRecord is called, it creates a new record internally, so Patient owns its records, hence its also a composition relationship.

PatientRecords is just a data holder: measurement value, type, patient ID, timestamp, and getters.

DataReader is an interface defining how data enters the system. Any class that implements it must provide a readData() method that takes DataStorage and throws IOExeption. It's included here because it's the entry point into storage. The actual reader classes for TCP, WebSocket, or file input belong to the Data Access Layer subsystem.

Patient Identification system



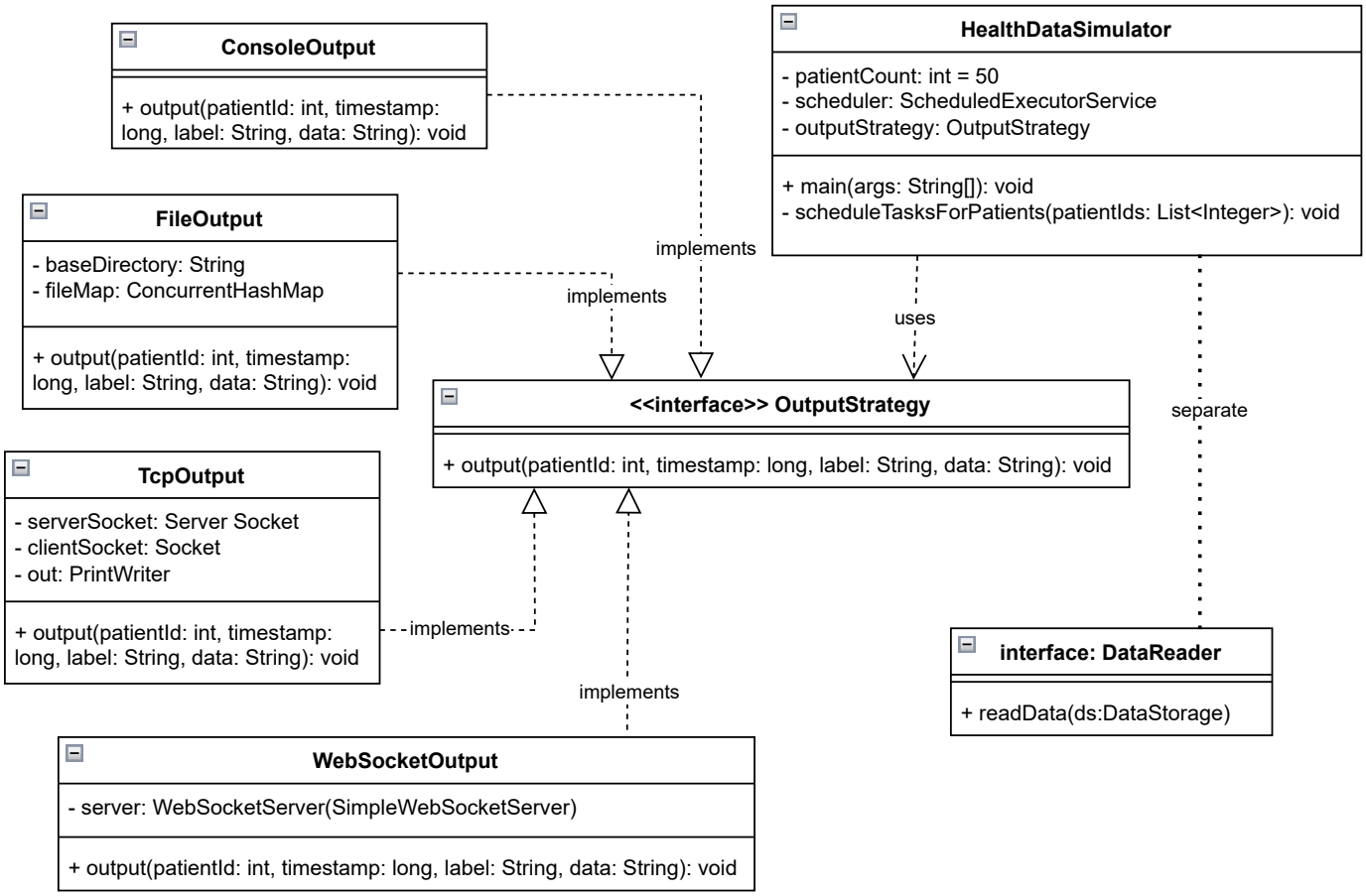
This diagram models how the system makes sure internal data is linked to the right patient.

PatientIdentifier takes an incoming patient ID, checks whether it's known, and tries to match it to a real hospital patient. If something doesn't match, handleMismatch() deals with it privately.

IdentityManager handles the actual identity logic by keeping a map of IDs to patients, logging any suspicious activity in anomalyLog, and running integrity checks across the system. Splitting this from PatientIdentifier keeps the two responsibilities (coordination and identity logic) cleanly separated.

HospitalPatient represents a patient as known to the hospital database, with a name, hospital ID, and medical history. It's linked to the simulator's Patient class by patient ID via an association. They represent the same real person in two different contexts. DataStorage is included as a stub because PatientIdentifier needs to look up patient records to confirm a match. Privacy is considered as the medicalHistory field is private and only accessible through controlled methods.

Data Access Layer



This diagram models how external data gets into the system. The design is built around the OutputStartegy interface, which has a single ouptut method.

All four concrete classes (TcpOutputStrategy, FileOutputStrategy, ConsoleOutputStrategy, and WebSocketOutputStrategy) implement OutputStrategy interface. They all do the same job but send data to different destinations: a TCP socket, local files, the console, or a WebSocket connection.

HealthDataSimulator holds one OutputStrategy reference and sends all data through it.

The specific strategy is chosen at startup via command line. This means swapping output destinations requires no changes to the simulator itself.

DataReader is included to show the input side of this layer. It's an interface for reading data into storage, mirroring the flexibility of OutputStrategy on the input side.

There are two separate classes named AlertGenerator in the codebase one in com.alerts that evaluates clinical conditions, and one in com.cardio_generator.generators used by HealthDataSimulator that generates random alert events for simulation. They are completely unrelated despite sharing a name.